



APPENDIX F

Code Samples



Any fool can write code that a computer can understand. Good programmers write code that humans can understand.

— *Martin Fowler* —



Code Sample F.1: The TAKAHE function `integrate_eoms()`

```

1 def integrate_eoms(a0, e0, p):
2     """
3     General purpose integrator for Nyadzani & Razzaque eqns for a & e.
4
5     Params:
6         a0 - The initial semimajor axis, measured in solar radii
7         e0 - The initial eccentricity, dimensionless
8         p - A vector of parameters:
9             p[1] = m1 (units: Solar Mass)
10            p[2] = m2 (units: Solar Mass)
11            p[3] = unused (will be removed in next version)
12            p[4] = unused (will be removed in next version)
13            p[5] = Lifetime (evolution + rejuvenation)
14
15     Returns:
16         A - An array of the semimajor axes of the binary system over time. (Solar
17            ↪ Mass)
18         E - An array of the eccentricities of the binary system over time. (no
19            ↪ dim.)
20     """
21
22     Solar_Mass = 1.989e30          # kg
23     Solar_Radius = 696340000.0    # m
24     G = 6.67e-11                  # m^3 kg^-1 s^-2
25     c = 299792458.0              # m/s
26
27     A = np.array([])
28     E = np.array([])
29     H = np.array([])
30
31     m1, m2, _, _, evotime = p[0], p[1], p[2], p[3], p[4]
32
33     # number of seconds in a year
34     seconds_per_year = 60 * 60 * 24 * 365.25
35
36     #####
37     #      Unit Check      #
38     #####
39     a = a0 * Solar_Radius    # Meters
40     e = e0                    # Dimensionless
41     m1 = m1 * Solar_Mass     # Kilogram
42     m2 = m2 * Solar_Mass     # Kilogram
43     #####
44     #      End Unit Check      #
45     #####
46
47     # Beta has units m^4 / s
48     beta = ((64/5) * G**3 * m1 * m2 * (m1 + m2) / (c**5))
49
50     A = np.append(A, a)
51     E = np.append(E, e)
52     H = np.append(H, 0.0)
53
54     total_time = 0
55
56     # Integrate until past the end of the universe, or a 10km orbit
57     while total_time/seconds_per_year + evotime < 1e11 and a > 1e4:
58         # an euler integrator: work out da/dt then times it by dt
59         # to get da, which then we can work out as a = a + da/dt * dt.

```

```

58     initial_da = (- beta / ((a**3) * (1 - e**2)**(7/2)))
59     da = initial_da * (1 + (73/24) * e**2 + (37/96) * e**4)
60
61     initial_de = (((-19/12) * beta) / (a**4*(1-e**2)**(5/2)))
62     de = initial_de * (e + (121/304) * e**3)
63     # Units: s^-1
64
65     timeA = abs(1e-2 * a/da)
66
67     if e > 1e-10:
68         timeE = abs(1e-2 * e/de)
69     else:
70         de = 0
71         e = 1e-10
72         timeE = timeA * 10
73
74     # maximum timestep is half of the width of the smallest BPASS time bin
75     conv_frac = 0.23076752*0.5*seconds_per_year
76
77     dt2 = (evotime + total_time/seconds_per_year)*conv_frac
78
79     # Take a timestep that results in the smallest change: either a
80     # change in E, a change in A, 1/2 the smallest BPASS bin.
81     dt = min(timeE, timeA, dt2)
82
83     a = a + dt * da
84     e = e + dt * de
85
86     A = np.append(A, a)
87     E = np.append(E, e)
88     H = np.append(H, dt)
89
90     total_time = total_time + dt
91
92     # Determine why we stopped, so the parent function can tell what went
93     # "wrong".
94     stop_reason = "flag_not_set"
95
96     if total_time/seconds_per_year + evotime >= 1e11:
97         stop_reason = "out_of_time"
98
99     if a <= 1e4:
100         stop_reason = "merged"
101
102     # Solar Radii, Dimensionless
103     return A / Solar_Radius, E, H, stop_reason

```

Code Sample F.2: The TAKAHE function integrate_timescale()

```

1 def integrate_timescale(m1, m2, p0, e0):
2     """
3     General purpose integrator for eqn(5.14) of Peters, 1964 [1].
4
5     [1] https://ui.adsabs.harvard.edu/abs/1964PhRv..136.1224P/abstract
6
7     Params:
8         m1 -- The mass of the first star (Solar Mass)
9         m2 -- The mass of the second star (Solar Mass)
10        p0 -- The initial period (days)

```

```

11     e0 -- The initial eccentricity
12
13 Returns:
14     tC -- The coalescence time          (seconds)
15     """
16 Solar_Mass = 1.989e30          # kg
17 Solar_Radius = 696340000.0    # m
18 G = 6.67e-11                 # m^3 kg^-1 s^-2
19 c = 299792458.0              # m/s
20
21 m1 = float(m1)
22 m2 = float(m2)
23 p0 = float(p0)
24 e0 = float(e0)
25
26 #####
27 # BEGIN UNIT CHECK #
28 #####
29 m1 = m1 * Solar_Mass          # Kilograms    ##
30 m2 = m2 * Solar_Mass          # Kilograms    ##
31 p0 = p0 * 24 * 60 * 60        # Seconds      ##
32 e0 = e0                       # Dimensionless ##
33 #####
34 # END UNIT CHECK #
35 #####
36
37 # Use Kepler's Third Law to compute the semimajor axis, in meters
38 a0 = (p0**2.0 * (G * (m1+m2)) / (4.0 * np.pi**2.0))**(1.0/3.0) # meters
39
40 # compute the constant c0
41 c0 = a0 * (1.0-e0**2.0) * e0**(-12.0/19.0) * (1.0+(121.0 / 304.0 *
42 ↪ e0**2.0))**(-870.0/2299.0) # meters
43
44 # Compute the constant beta
45 beta = 64.0 / 5.0 * G**3.0 * m1 * m2 * (m1 + m2) / c**5.0
46 # meters^4 / s
47
48 # Circular binary coalescence time
49 tC = a0**4.0 / (4.0*beta)
50
51 if e0 != 0:
52     if e0 < 0.01:
53         # Low ecc - see eqn after eqn(5.14) of Peters, 1964
54         tC = c0**4.0 * e0**(48.0/19.0) / (4.0*beta)
55     elif e0 > 0.99:
56         # High ecc - see eqn after eqn after eqn(5.14) of Peters, 1964.
57         tC = tC * ((768.0 / 425.0) * (1.0-e0**2.0)**3.5)
58     else:
59         # Medium ecc - see eqn(5.14) of Peters, 1964
60         e = 0.0
61         de = e0 / 10000.0
62         summand = 0.0
63
64         while e < e0:
65             this_integral = de * e**(29.0/19.0)
66             this_integral *= (1.0 + (121.0/304.0) * e**2.0)**(1181.0/2299.0)
67             this_integral /= (1.0 - e**2.0)**(1.5)
68
69             summand += this_integral
70             e += de

```

```
70
71         e = float(e)
72
73         tC = (12.0/19.0) * (c0**4.0 / beta) * summand
74
75     return tC
```

